SOLID PRINCIPLES

SOLID principles are five essential guidelines that enhance software design, making code more maintainable and scalable.

Helps in enhancing loose coupling. Loose coupling means a group of classes are less dependent on one another. Helps in making code more reusable, maintainable, flexible and stable. Loosely coupled classes minimize changes in your code when some changes are required in some other code.

SOLID

S – Single Responsibility Principle

O – Open Closed Principle

L – Liskov Substitution Principle

I – Interface Segregation Principle

D – Dependency Inversion Principle

NEED FOR SOLID PRINCIPLES

- SOLID principles make code easier to maintain. When each class has a clear responsibility, it's simpler to find where to make changes without affecting unrelated parts of the code.
- These principles support growth in software. For example, the Open/Closed Principle allows developers to add new features without changing existing code, making it easier to adapt to new requirements.
- SOLID encourages flexibility. By depending on abstractions rather than specific implementations (as in the Dependency Inversion Principle), developers can change components without disrupting the entire system

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

This principle states that "**A class should have only one reason to change**" which means every class should have a single responsibility or single job or single purpose. In other words, a class should have only one job or purpose within the software system.

Ex: Imagine a baker who is responsible for baking bread. The baker's role is to focus on the task of baking bread, ensuring that the bread is of high quality, properly baked, and meets the bakery's standards.

```
class BreadBaker
{
   public void bakeBread()
   {
      System.out.println("Baking high-quality bread...");
   }
}

class InventoryManager
{
   public void manageInventory()
   {
      System.out.println("Managing inventory...");
   }
}

class SupplyOrder
{
   public void orderSupplies()
   {
      System.out.println("Ordering supplies...");
   }
}
```

```
class CustomerService
{
   public void serveCustomer()
   {
      System.out.println("Serving customers...");
   }
}

class BakeryCleaner
{
   public void cleanBakery()
   {
      System.out.println("Cleaning the bakery...");
   }
}

public class SRP1
{
   public static void main(String[] args) {
      {
         BreadBaker baker = new BreadBaker();
         InventoryManager inventoryManager = new InventoryManager();
         SupplyOrder supplyOrder = new SupplyOrder();
         CustomerService customerService = new CustomerService();
         BakeryCleaner cleaner = new BakeryCleaner();

         //Each class focuses on its specific responsibility
         baker.bakeBread();
         inventoryManager.manageInventory();
```

```
            supplyOrder.orderSupplies();
            customerService.serveCustomer();
            cleaner.cleanBakery();
        }
    }
}
```

Output:

```
Baking high-quality bread...
Managing inventory...
Ordering supplies...
Serving customers...
Cleaning the bakery...
```

Without SRP (Bad Design)

```
//Violates SRP: One class has multiple responsibilities
class Invoice {
    private String item;
    private int quantity;
    private double pricePerItem;

    public Invoice(String item, int quantity, double pricePerItem) {
        this.item = item;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public double calculateTotal() {
```

```
        return quantity * pricePerItem;
    }

    // Responsibility 1: Persistence
    public void saveToFile() {
        System.out.println("Saving invoice of " + item + " to file...");
    }

    // Responsibility 2: Printing
    public void printInvoice() {
        System.out.println("Invoice Details:");
        System.out.println(item + " x " + quantity + " @ " + pricePerItem + " each");
        System.out.println("Total: " + calculateTotal());
    }
}

public class SRPViolationDemo {
    public static void main(String[] args) {
        Invoice invoice = new Invoice("Laptop", 2, 55000.0);
        invoice.printInvoice();
        invoice.saveToFile();
    }
}
```

Output:

```
Invoice Details:
Laptop x 2 @ 55000.0 each
Total: 110000.0
Saving invoice of Laptop to file...
```

Problem – Invoice is doing three things: Holding invoice data, handling persistence, and printing

With SRP (Good design)

We separate each responsibility into three different classes

```
// Holds invoice data + calculation
class Invoice {
    private String item;
    private int quantity;
    private double pricePerItem;

    public Invoice(String item, int quantity, double pricePerItem) {
        this.item = item;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getItem() { return item; }
    public int getQuantity() { return quantity; }
    public double getPricePerItem() { return pricePerItem; }

    public double calculateTotal() {
        return quantity * pricePerItem;
    }
}

// Handles persistence (saving to file/db)
class InvoiceRepository {
    public void saveToFile(Invoice invoice) {
```

```
        System.out.println("Saving invoice of " + invoice.getItem() + " to file...");
    }
}

// Handles printing
class InvoicePrinter {
    public void print(Invoice invoice) {
        System.out.println("Invoice Details:");
        System.out.println(invoice.getItem() + " x " + invoice.getQuantity() +
                           " @ " + invoice.getPricePerItem() + " each");
        System.out.println("Total: " + invoice.calculateTotal());
    }
}

// Demo
public class SRPExampleDemo {
    public static void main(String[] args) {
        Invoice invoice = new Invoice("Laptop", 2, 55000.0);

        InvoicePrinter printer = new InvoicePrinter();
        printer.print(invoice);

        InvoiceRepository repository = new InvoiceRepository();
        repository.saveToFile(invoice);
    }
}
```

Output:

```
Invoice Details:
Laptop x 2 @ 55000.0 each
```

Total: 110000.0

Saving invoice of Laptop to file...

- Invoice → only holds invoice data & calculation.
- InvoicePrinter → only responsible for displaying/printing.
- InvoiceRepository → only responsible for saving.

OPEN CLOSED PRINCIPLE (OCP)

This principle states that “Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification” which means you should be able to extend a class behavior, without modifying it.

Ex: Imagine you have a class called PaymentProcessor that processes payments for an online store. Initially, the PaymentProcessor class only supports processing payments using credit cards. However, you want to extend its functionality to also support processing payments using PayPal.

```
// Base class for payment processing (Open for extension, closed for modification)

abstract class PaymentProcessor
{
   public abstract void processPayment(double amount);
}

class CreditCard extends PaymentProcessor
{
   @Override
   public void processPayment(double amount)
   {
      System.out.println("Processing credit card payment of $" + amount);
   }
}

class PayPal extends PaymentProcessor
{
   @Override
   public void processPayment(double amount)
   {
      System.out.println("Processing Paypal payment of $" + amount);
```

```
    }
}

public class OCP1
{
    public static void main(String[] args)
    {
        PaymentProcessor cc = new CreditCard();
        PaymentProcessor pp = new PayPal();

        processPayment(cc, 100.00); //Processing credit card payment
        processPayment(pp, 150.00); //Processing paypal payment
    }

    // Generic method that works for any payment processor
    public static void processPayment(PaymentProcessor processor, double amount)
    {
        processor.processPayment(amount);
    }
}
```

Output:

Processing credit card payment of $100.0

Processing Paypal payment of $150.0

Principle Recap:

1. Open for extension – Able to add new functionality without changing existing code
2. Closed for modification – Existing, tested, working code should not be changed when requirements evolve.

1. Base Class (Abstraction)

```
abstract class PaymentProcessor
{
   public abstract void processPayment(double amount);
}
```

- Defines the **common contract** (processPayment) for all payment processors.
- It does not care *how* the payment is processed.
- This class is **closed for modification** → We don't need to change it when adding new payment types.

2. Concrete Implementations

```
class CreditCardPaymentProcessor extends PaymentProcessor
{
   @Override
   public void processPayment(double amount)
{
      System.out.println("Processing credit card payment of $" + amount);
   }
}

class PayPalPaymentProcessor extends PaymentProcessor
{
   @Override
   public void processPayment(double amount)
{
      System.out.println("Processing PayPal payment of $" + amount);
   }
}
```

- Each class **extends** the base class and **implements the payment logic** differently.
- If tomorrow you need a **UPI** or **Bitcoin** processor, you just create a new class like this:

```
class UPIPaymentProcessor extends PaymentProcessor
{
        @Override
         public void processPayment(double amount)
         {
             System.out.println("Processing UPI payment of $" + amount);
         }
}
```

No change in existing classes → **Open for extension**.

Why this is OCP?

Closed for modification: We don't need to modify existing tested classes (PaymentProcessor, CreditCard, PayPal, or Main) when we add new payment methods.

If tomorrow we add BitCoin, existing code remains untouched.

Another Example: Notification System

Problem Statement

Imagine you're building a **Notification System** for a social media app.

- Initially, the app only supports **Email notifications**.
- Later, requirements change: you also need **SMS notifications**, and then maybe **Push notifications**.

If we write all logic inside a single class using if-else, every time we add a new type of notification, we must **modify existing code** → violating **OCP**.

Instead, we'll use **Open/Closed Principle**:

- Base class: Notifier (abstract contract).
- Extensions: EmailNotifier, SMSNotifier, PushNotifier.
- Client code (NotificationService) remains unchanged when we add new notification types.

```
// Abstract base class (Closed for modification, Open for extension)

abstract class Notifier
{
   public abstract void send(String message);
}

// Concrete implementation: Email

class EmailNotifier extends Notifier
{
   @Override
   public void send(String message)
   {
      System.out.println("Sending Email: " + message);
   }
}

// Concrete implementation: SMS

class SMSNotifier extends Notifier
{
   @Override
   public void send(String message)
   {
      System.out.println("Sending SMS: " + message);
   }
}

// Concrete implementation: Push Notification
```

```
class PushNotifier extends Notifier
{
    @Override
    public void send(String message)
    {
        System.out.println("Sending Push Notification: " + message);
    }
}

// Client class that uses any notifier

class NotificationService
{
    public void notifyUser(Notifier notifier, String message)
    {
        notifier.send(message);
    }
}

// Main class

public class OCP2
{
    public static void main(String[] args)
    {
        NotificationService service = new NotificationService();

        Notifier email = new EmailNotifier();
        service.notifyUser(email, "Welcome to our app!");
```

```
        Notifier sms = new SMSNotifier();
        service.notifyUser(sms, "Your OTP is 123456");

        Notifier push = new PushNotifier();
        service.notifyUser(push, "You have a new friend request!");
    }
}
```

Output:

Sending Email: Welcome to our app!

Sending SMS: Your OTP is 123456

Sending Push Notification: You have a new friend request!

How this follows OCP?

1. **Closed for modification**
    - NotificationService and Notifier base class never need changes.
    - Tested existing code remains untouched.
2. **Open for extension**
    - If tomorrow we add WhatsAppNotifier or SlackNotifier, we just **create a new class** extending Notifier.
    - Client (NotificationService) still works without modification.

LISKOV SUBSTITUTION PRINCIPLE

The principle was introduced by Barbara Liskov in 1987 and according to this principle "Derived or child classes must be substitutable for their base or parent classes". This principle ensures that any class that is the child of a parent class should be usable in place of its parent without any unexpected behaviour.

Ex: One of the classic examples of this principle is a rectangle having four sides. A rectangle's height can be any value and width can be any value. A square is a rectangle with equal width and height. So we can say that we can extend the properties of the rectangle class into square class.

```
interface Shape
{
   double area();
}

class Rectangle implements Shape
{
   private double width;
   private double height;

   public Rectangle(double w, double h)
   {
      this.width = w;
      this.height = h;
   }

   @Override
   public double area()
   {
      return width * height;
   }
```

```
}

class Square implements Shape
{
    private double side;

    public Square(double side)
    {
        this.side = side;
    }

    @Override
    public double area()
    {
        return side * side;
    }
}

public class LSP1
{
    public static void main(String[] args)
    {
        Shape rect = new Rectangle(10, 20);
        Shape sq = new Square(15);

        System.out.println("Rectangle Area: " + rect.area());
        System.out.println("Square Area: " + sq.area());
    }
}
```

Output:

Rectangle Area: 200.0

Square Area: 225.0

Why this follows LSP?

- Both Rectangle and Square are substitutable as Shape.
- Client code (LSP1) only depends on the Shape interface.
- No unexpected behavior: each class handles its own rules internally.
- We avoided inheritance where it breaks logical expectations.

Ex: Document management system

- We have a base class Document with a method printContent().
- Different document types: PDFDocument, WordDocument, TextDocument.
- Any subclass of Document should be usable in place of the base class **without breaking functionality**.

If we wrongly design a class (e.g., ReadOnlyDocument that throws exception when asked to print), it would **violate LSP**, because a ReadOnlyDocument object can't behave like a Document.

So, to follow **LSP**, each subclass must **honor the behavior** promised by the base class.

```
// Base class: Document
abstract class Document {
    protected String content;

    public Document(String content) {
        this.content = content;
    }

    // All subclasses must support printing
    public abstract void printContent();
}
```

```
// PDF document
class PDFDocument extends Document {
    public PDFDocument(String content) {
        super(content);
    }

    @Override
    public void printContent() {
        System.out.println("Printing PDF Document: " + content);
    }
}

// Word document
class WordDocument extends Document {
    public WordDocument(String content) {
        super(content);
    }

    @Override
    public void printContent() {
        System.out.println("Printing Word Document: " + content);
    }
}

// Text document
class TextDocument extends Document {
    public TextDocument(String content) {
        super(content);
    }
```

```
    @Override
    public void printContent() {
        System.out.println("Printing Text Document: " + content);
    }
}

// Client code that works with base class
class DocumentPrinter {
    public void printDocument(Document doc) {
        doc.printContent(); // Works for any subclass
    }
}

// Demo
public class LSPDemo {
    public static void main(String[] args) {
        DocumentPrinter printer = new DocumentPrinter();

        Document pdf = new PDFDocument("Java OOP Principles");
        Document word = new WordDocument("Liskov Substitution Principle");
        Document text = new TextDocument("Hello, World!");

        // Subclasses can be substituted without breaking code
        printer.printDocument(pdf);
        printer.printDocument(word);
        printer.printDocument(text);
    }
}
```

Output:

Printing PDF Document: Java OOP Principles

Printing Word Document: Liskov Substitution Principle

Printing Text Document: Hello, World!

How this shows LSP

- **Base class (Document)** promises that every document can printContent().
- **Subclasses (PDFDocument, WordDocument, TextDocument)** honor that contract.
- In DocumentPrinter, we can replace Document with any subclass and the program works correctly.
- No exceptions, no unexpected behavior → ✅ **LSP is followed**.

What would violate LSP?

Imagine we added this class:

```
class EncryptedDocument extends Document {
    public EncryptedDocument(String content) {
        super(content);
    }

    @Override
    public void printContent() {
        throw new UnsupportedOperationException("Cannot print encrypted document!");
    }
}
```

Now if DocumentPrinter receives an EncryptedDocument, it breaks → ❌ **LSP violation**, because the subclass isn't substitutable for the base class.

INTERFACE SEGREGATION PRINCIPLE (ISP)

This principle is the first principle that applies to interfaces instead of classes in SOLID and it is similar to the single responsibility principle. It states that “do not force any client to implement an interface which is irrelevant to them”.

Here your main goal is to focus on avoiding fat interface and give preference to many small client-specific interfaces. You should prefer many client interfaces rather than one general interface and each interface should have a specific responsibility.

Ex: Suppose if you enter a restaurant and you are pure vegetarian. The waiter in that restaurant gave you the menu card which includes vegetarian items, non-vegetarian items, drinks and sweets.

- *In this case, as a customer, you should have a menu card which includes only vegetarian items, not everything which you don't eat in your food. Here the menu should be different for different types of customers.*
- *The common or general menu card for everyone can be divided into multiple cards instead of just one. Using this principle helps in reducing the side effects and frequency of required changes.*

```
import java.util.*;


// Interface for vegetarian menu
interface IVegetarianMenu
{
   List<String> getVegetarianItems();
}


// Interface for non-vegetarian menu
interface INonVegetarianMenu
{
   List<String> getNonVegetarianItems();
}


// Interface for drinks menu
interface IDrinkMenu
```

```
{
    List<String> getDrinkItems();
}

// Class for vegetarian menu
class VegetarianMenu implements IVegetarianMenu
{
    public List<String> getVegetarianItems()
    {
        return Arrays.asList("Vegetable Curry", "Paneer Tikka", "Salad");
    }
}

// Class for non-vegetarian menu
class NonVegetarianMenu implements INonVegetarianMenu
{
    public List<String> getNonVegetarianItems()
    {
        return Arrays.asList("Chicken Curry", "Fist Fry", "Mutton Biryani");
    }
}

// Class for drinks menu
class DrinkMenu implements IDrinkMenu
{
    public List<String> getDrinkItems()
    {
        return Arrays.asList("Water", "Soda", "Juice");
    }
}
```

```
// Function to display menu items for a vegetarian customer
class MenuDisplay
{
    public static void displayVegetarianMenu(IVegetarianMenu menu)
    {
        System.out.println("Vegetarian Menu:");
        for (String item : menu.getVegetarianItems())
        {
            System.out.println("- " + item);
        }
    }

    public static void displayNonVegetarianMenu(INonVegetarianMenu menu)
    {
        System.out.println("Non-Vegetarian Menu:");
        for (String item : menu.getNonVegetarianItems())
        {
            System.out.println("- " + item);
        }
    }
}

public class ISP1
{
    public static void main(String[] args)
    {
        VegetarianMenu vegMenu = new VegetarianMenu();
        NonVegetarianMenu nonVegMenu = new NonVegetarianMenu();
```

```
        MenuDisplay.displayVegetarianMenu(vegMenu);
        MenuDisplay.displayNonVegetarianMenu(nonVegMenu);
    }
}
```

Output:

Vegetarian Menu:

- Vegetable Curry
- Paneer Tikka
- Salad

Non-Vegetarian Menu:

- Chicken Curry
- Fist Fry
- Mutton Biryani

Ex: Smart Home System

- *Initially, we might think of having one big SmartDevice interface with methods:*
    - *turnOn()*
    - *turnOff()*
    - *setTemperature()*
    - *playMusic()*
    - *recordVideo()*

⚠️ *Problem: Not all devices need all these methods.*

- *A **LightBulb** should not implement setTemperature() or playMusic().*
- *A **Thermostat** should not implement playMusic() or recordVideo().*
- *A **SmartSpeaker** should not implement setTemperature().*

👉 *If we force all devices to implement a "fat interface," they'll end up with **empty or irrelevant methods**, violating **ISP**.*

✅ Applying ISP (Solution)

Instead of one big interface, we split it into **small, specific interfaces**:

- Switchable → for devices that can be turned on/off.
- TemperatureControllable → for thermostats.
- MusicPlayable → for speakers.
- Recordable → for cameras.

Each device implements **only the interfaces it needs**.

```
interface Switchable
{
   void turnOn();
   void turnOff();
}

interface TemperatureControllable
{
   void setTemperature(int temperature);
}

interface MusicPlayable
{
   void playMusic(String song);
}

interface Recordable
{
   void recordVideo();
}

// LightBulb only cares about turning on/off
```

```
class LightBulb implements Switchable
{
    @Override
    public void turnOn()
    {
        System.out.println("LightBulb is ON");
    }

    @Override
    public void turnOff()
    {
        System.out.println("LightBulb is OFF");
    }
}

// Thermostat cares about temperature + switching
class Thermostat implements Switchable, TemperatureControllable
{
    private int temperature;

    @Override
    public void turnOn()
    {
        System.out.println("Thermostat is ON");
    }

    @Override
    public void turnOff()
    {
        System.out.println("Thermostat is OFF");
```

```
    }

    @Override
    public void setTemperature(int temperature)
    {
        this.temperature = temperature;
        System.out.println("Thermostat set to " + temperature + " degree Celsius");
    }
}

//Smart speaker cares about music + switching
class SmartSpeaker implements Switchable, MusicPlayable
{
    @Override
    public void turnOn()
    {
        System.out.println("SmartSpeaker is ON");
    }

    @Override
    public void turnOff()
    {
        System.out.println("SmartSpeaker is OFF");
    }

    @Override
    public void playMusic(String song)
    {
        System.out.println("Playing song: " + song);
    }
```

```
}

// Security Camera cares about recording + switching
class SecurityCamera implements Switchable, Recordable
{
    @Override
    public void turnOn()
    {
        System.out.println("SecurityCamera is ON");
    }

    @Override
    public void turnOff()
    {
        System.out.println("SecurityCamera is OFF");
    }

    @Override
    public void recordVideo()
    {
        System.out.println("Recording video...");
    }
}

public class ISP2
{
    public static void main(String[] args)
    {
        Switchable bulb = new LightBulb();
        Switchable thermostat = new Thermostat();
```

```
        Switchable speaker = new SmartSpeaker();
        Switchable camera = new SecurityCamera();

        bulb.turnOn();
        bulb.turnOff();

        ((Thermostat) thermostat).turnOn();
        ((Thermostat) thermostat).setTemperature(24);

        ((SmartSpeaker) speaker).turnOn();
        ((SmartSpeaker) speaker).playMusic("Shape of You");

        ((SecurityCamera) camera).turnOn();
        ((SecurityCamera) camera).recordVideo();
    }
}
```

Output:

LightBulb is ON

LightBulb is OFF

Thermostat is ON

Thermostat set to 24 degree Celsius

SmartSpeaker is ON

Playing song: Shape of You

SecurityCamera is ON

Recording video...

DEPENDENCY INVERSION PRINCIPLE

The Dependency Inversion Principle (DIP) is a principle in object-oriented design that states that "High-level modules should not depend on low-level modules. Both should depend on abstraction". Additionally, abstractions should not depend on details. Details should depend on abstractions.

- In simpler terms, the DIP suggests that classes should rely on abstractions (e.g., interfaces or abstract classes) rather than concrete implementations.
- This allows for more flexible and decoupled code, making it easier to change implementations without affecting other parts of the codebase.

Ex: In a software development team, developers depend on an abstract version control (e.g.., Git) to manage and track changes to the codebase. They don't depend on specific details of how Git works internally.

```
// interface for version control system
interface IVersionControl
{
   void commit(String message);
   void push();
   void pull();
}

// Git version control implementation
class GitVersionControl implements IVersionControl
{
   @Override
   public void commit(String message)
   {
      System.out.println("Committing changes to Git with message: " + message);
   }

   @Override
   public void push()
```

```
    {
        System.out.println("Pushing changes to remote Git repository");
    }

    @Override
    public void pull()
    {
        System.out.println("Pulling changes from remote Git repository");
    }
}

// Team class that relies on version control
class DevelopmentTeam
{
    private IVersionControl versionControl;

    public DevelopmentTeam(IVersionControl vc)
    {
        this.versionControl = vc;
    }

    public void makeCommit(String message)
    {
        versionControl.commit(message);
    }

    public void perforPush()
    {
        versionControl.push();
    }
```

```
    public void performPull()
    {
        versionControl.pull();
    }
}

public class DIP1
{
    public static void main(String[] args)
    {
        GitVersionControl git = new GitVersionControl();
        DevelopmentTeam team = new DevelopmentTeam(git);

        team.makeCommit("Initial Commit...");
        team.performPull();
        team.perforPush();
    }
}
```

Output:

Committing changes to Git with message: Initial Commit...

Pulling changes from remote Git repository

Pushing changes to remote Git repository

Explanation:

- **IVersionControl Interface**: This defines the operations that any version control system should support, like commit, push, and pull. It serves as an abstraction that decouples high-level code from low-level implementations.
- **GitVersionControl Class**: This class implements the IVersionControl interface, providing specific functionality for managing version control using Git.

- **DevelopmentTeam Class**: This class relies on the IVersionControl interface, meaning it can work with any version control implementation that adheres to the interface. It does not need to know the details of how Git works internally.

Ex: NEWS Application

- *The app can fetch news from different sources: **API**, **Database**, or maybe **File storage**.*
- *Without DIP, the **high-level module (NewsReader)** directly depends on low-level modules (APINewsService, DatabaseNewsService), making it **hard to extend or change data sources**.*

👉 ***DIP says:***

- *High-level modules should not depend on low-level modules.*
- *Both should depend on **abstractions (interfaces)**.*

❌ Without DIP (Bad Design)

```
class APINewsService {
    public String getNews() {
        return "News from API";
    }
}

class NewsReader {
    private APINewsService apiService = new APINewsService();

    public void showNews() {
        System.out.println(apiService.getNews());
    }
}

public class DIPViolationDemo {
    public static void main(String[] args) {
        NewsReader reader = new NewsReader();
        reader.showNews(); // Works only with API, tightly coupled!
```

```
    }
}
```

Problem: NewsReader is **tightly coupled** to APINewsService. If tomorrow we want **Database** or **File storage**, we must **modify NewsReader** → violates DIP.

✅ With DIP (Good Design)

```
interface NewsService
{
    String getNews();
}

class APINewsService implements NewsService
{
    @Override
    public String getNews()
    {
        return "News from API";
    }
}

class DatabaseNewsService implements NewsService
{
    @Override
    public String getNews()
    {
        return "News from Database";
    }
}
```

```
class FileNewsService implements NewsService
{
    @Override
    public String getNews()
    {
        return "News from File Storage";
    }
}

class NewsReader
{
    private NewsService ns;

    // Constructor injection (DIP in action)
    public NewsReader(NewsService ns)
    {
        this.ns = ns;
    }

    public void showNews()
    {
        System.out.println(ns.getNews());
    }
}

public class DIP2
{
    public static void main(String[] args)
    {
        NewsService api = new APINewsService();
```

```
        NewsReader reader1 = new NewsReader(api);
        reader1.showNews();

        NewsService db = new DatabaseNewsService();
        NewsReader reader2 = new NewsReader(db);
        reader2.showNews();

        NewsService file = new FileNewsService();
        NewsReader reader3 = new NewsReader(file);
        reader3.showNews();
    }
}
```

Output:

News from API

News from Database

News from File Storage

🔑 **Why this follows DIP?**

1. NewsReader (high-level) doesn't depend on concrete classes.
2. Both high-level (NewsReader) and low-level (APINewsService, DatabaseNewsService, FileNewsService) depend on **abstraction (NewsService)**.
3. Adding a new source (e.g., CloudNewsService) doesn't require modifying NewsReader.